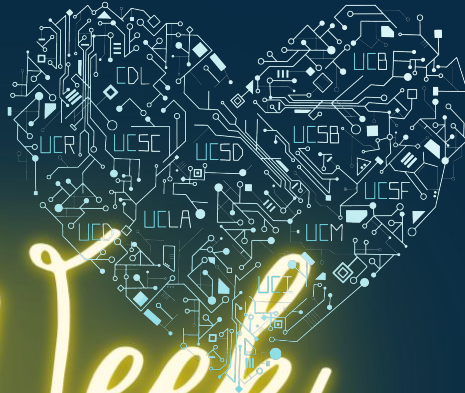




Love Data Week

February 9 - 13, 2026

UC LOVE DATA WEEK:

**A SERIES OF VIRTUAL TALKS, PRESENTATIONS, AND WORKSHOPS
BEING OFFERED AS A WEEK-LONG EVENT FOCUSED ON DATA
ACCESS, MANAGEMENT, SECURITY, SHARING, AND PRESERVATION,
FOR THE UC COMMUNITY.**

**ALL EVENTS ARE FREE TO ATTEND AND OPEN TO ANY MEMBER OF
THE UC COMMUNITY.**

**FOR MORE INFORMATION AND TO REGISTER FOR A
SPECIFIC WORKSHOP, PLEASE VISIT OUR WEBSITE:**

<https://uc-love-data-week.github.io/>



Love Data Week

Code of Conduct

By participating in this community, participants accept to abide by the UC Love Data Week (UCLDW) Code of Conduct and accept the procedures by which any Code of Conduct incidents are resolved. Any form of behavior to exclude, intimidate, or cause discomfort is a violation of the Code of Conduct.

Expected Behavior

- Use welcoming and inclusive language
- Be respectful of different viewpoints and experiences
- Gracefully provide and accept constructive criticism
- Focus on what is best for the community
- Show courtesy and respect towards other community members

If you believe someone is violating the Code of Conduct, we ask that you report it to the instructor/helper in your session or to the UCLDW email address (uclovedataweek@gmail.com)

[Code of Conduct Full Document](#)

Making Research Software Citable & Discoverable

UC Love Data Week 2026

Reid Otsuji, *University of California San Diego*

Karla Padilla, *University of California San Diego*

Leigh Phan, *University of California Los Angeles*

Tim Dennis, *University of California Los Angeles*

Laura Langdon, *University of California OSPO Network*

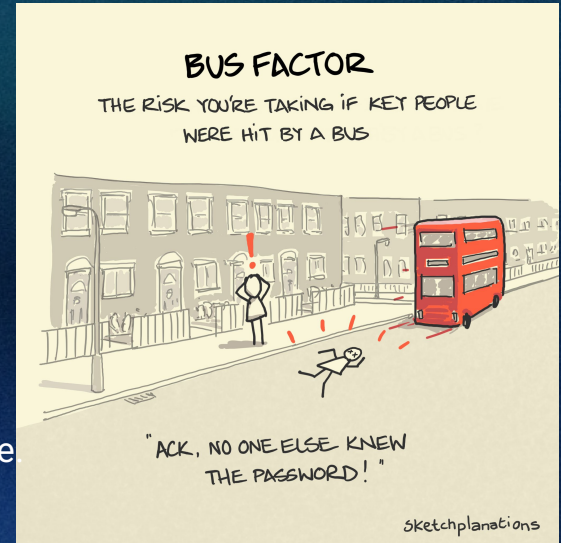
The "Works on My Machine" Trap

The Problem:

- You send your code to a colleague...
- "It crashes on line 1" 😞
- "I can't install the dependencies"
- "Which Python version did you use?"

The Bus Factor Problem:

Code works perfectly on ONE laptop. If it disappears, the research is gone.



The "Works on My Machine" Trap

```
$ python src/analysis.py
```

```
Traceback (most recent call last):
```

```
File "analysis.py", line 1, in <module>
```

```
    import numpy as np
```

```
ModuleNotFoundError: No module named 'numpy'
```

This is where most research software lives. Fragile.

What's the fix? FAIR4RS Principles →

FAIR4RS Principles

What We're Doing TODAY:

F - Findable: Software and metadata are easy to discover
→ DOI, CITATION.cff, metadata

A - Accessible: Retrievable via standard protocols
→ Public GitHub, git clone, Zenodo

I - Interoperable: Exchanges data with other tools
→ Standard formats, documented dependencies

R - Reusable: Can be executed AND modified
→ LICENSE, README, environment files



The "Works on My Machine" Trap

```
=== STAGE: The Before State (Branch: 01-start) ===  
--- DIRECTORY STRUCTURE ---  
./  
├─ environment.toml  <-- 'Just works for me'  
├─ README.md        <-- 'Just a title'  
└─ src/  
    └─ analysis.py   <-- The hidden logic  
  
2 directories, 3 files
```

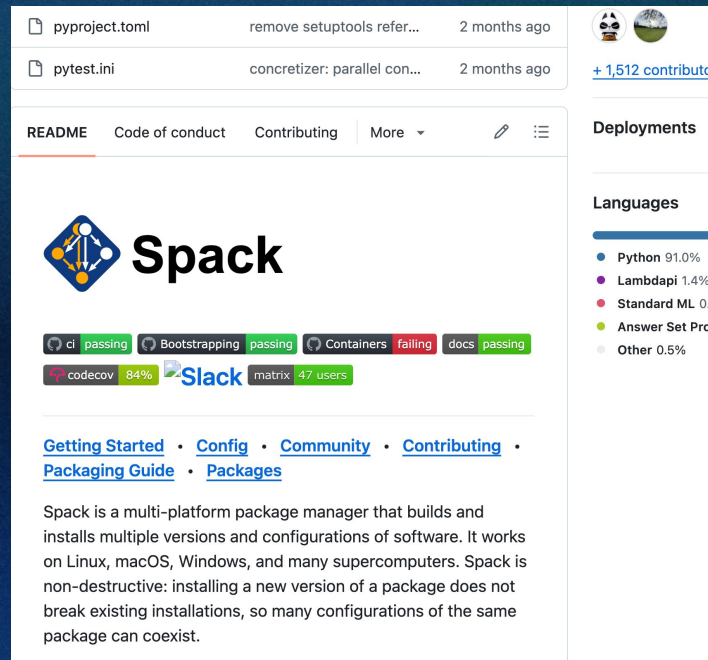
What's Missing?

✗ No LICENSE • ✗ No environment • ✗ No citation • ✗ No DOI

But before we fix it step-by-step, let's see what "done right" looks like in a real research project.

What Does "FAIR Research Software" Look Like?

- **Spack**: A production research software project done right
- Used by national labs, universities, supercomputing centers
- Check the **README**, **LICENSE**, **CITATION.cff**, documentation
- Notice what makes it discoverable and reusable
- Great examples:
<https://github.com/matiassingerg/awesome-readme>



From Fragile to FAIR

We've seen the problem: fragile, undocumented code.

We've seen the solution: Spack with all the pieces in place.

Now let's build it step-by-step:

- ✓ License - So others can legally use your work (Reid)
- ✓ Environment - So it runs on any machine
- ✓ Citation - So you get academic credit
- ✓ README - So people can find and use it

Let's start with the foundation: **Licensing**.

Why Licensing Matters?

- Clear licensing communicates permission and expectations, reducing friction and supporting open, reproducible research.
- But before choosing a license or making the decision to open source your software, it is important to determine who owns the code.

Licensing can be trickier than you would expect!

Why Licensing Matters?

When researchers publish code **without a license**, most people assume it is “public.”

- Legally, it is not.
- Copyright law applies automatically.

Without a license:

- others cannot reuse, modify, or redistribute the code
- collaboration becomes uncertain and risky
- the sustainability of the project suffers

Understanding license categories

Open-source licenses fall into two broad categories.

Permissive licenses:

- These are "no strings attached" licenses.
- Idea: "Just take it and use it; we don't care what you do next."

Copyleft licenses

- These are often called "reciprocal" or "viral" licenses.
- Idea: "We give you this freedom, but you must ensure the next person has the same freedom."

Permissive Licenses

Examples: BSD, MIT, Apache 2.0

Permissive licenses - have few requirements and are relatively easy to comply with. Typically, you are required to provide attribution.

These licenses allow users to distribute your software commercially

BSD and MIT are often preferred in UC environments due to their simplicity and lack of explicit patent grant provisions.

- The Apache license requires licensors to grant a patent license to all licensees.

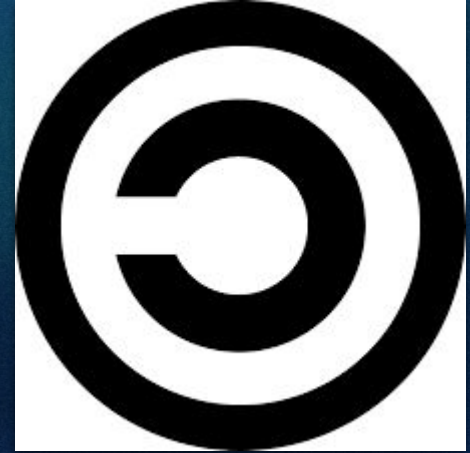


Copyleft Licenses

Examples: GNU General Public License 2.0 or GPL 2.0

Copyleft licenses - these licenses require that “derivative works/modifications” must be licensed under the same terms of the copyleft license.

This protects openness across the lifecycle of a project.



How to choose a license for your UC project

Quick reference

Your need	Recommended license	Why
Default / most projects	BSD 3-Clause	UC's standard recommendation
Simplest possible	MIT	Minimal text, very popular
Industry collaboration	Apache 2.0	Explicit patent terms
Educational focus	ECL 2.0	Education-specific variant

Before releasing code under an open-source license, you must check with your campus Tech Transfer office (sometimes called the Office of Intellectual Property or Technology Commercialization).

Selection of a license will depend on your goals, and the policies of the University of California. The [Open Source Software chart](#) and its companion can help you to navigate the UC policies and help guide your discussion with your TTO.

Always consult your campus **Tech Transfer office before releasing UC-owned software.**

When creating a new Github repository - Choose your License

The UC Standard Recommendation is:

BSD 3-Clause

Create a new repository

Repositories contain a project's files and version history. Have a project elsewhere? [Import a repository](#).
Required fields are marked with an asterisk (*).

1 **General**

Owner * U2NG / Repository name *
open-source-software-project is available.

Great repository names are short and memorable. How about [solid-giggle](#)?

Description

0 / 350 characters

2 **Configuration**

Choose visibility *
Choose who can see and commit to this repository

Start with a template
Templates pre-configure your repository with files.

Add README
READMEs can be used as longer descriptions. [About READMEs](#)

Add .gitignore
.gitignore tells git which files not to track. [About ignoring files](#)

Add license
Licenses explain how others can use your code. [About licenses](#)

Choose a license

- ☒ No license
- ☐ Apache License 2.0
- ☐ GNU General Public License v3.0
- ☐ MIT License
- ☐ BSD 2-Clause "Simplified" License
- ☒ **BSD 3-Clause "New" or "Revised" License**
- ☐ Boost Software License 1.0
- ☐ Creative Commons Zero v1.0 Universal
- ☐ Eclipse Public License 2.0
- ☐ GNU Affero General Public License

UC Licensing Resources

- [UC OSS Resources](#) – Resources and guides for open source activities at UC.
- [UC OSS Chart and Companion Guide](#) – The definitive list of UC-recommended "low-risk" licenses.
- [UC OSPO License Guide](#) – Guidance on institutional requirements and templates.
- [ChooseALicense.com](#) – A helpful tool created by GitHub for comparing license features. This tool does not supersede UC policies.

Why Environments Matter

Code rots. Python updates, packages break, and 6 months from now, your script won't run.

The Problem: Research software often "works on my machine" and nowhere else.

Different operating systems, outdated packages, and mismatched library versions frequently break code.

What Environment Management Captures:

- The exact language versions used
- Required packages
- The dependency set needed to run the software
- Instructions for reproducing the execution environment

The Payoff: We aren't just shipping code; we're shipping the **computer state** needed to run it.



Pixi

POWERFUL WORKFLOW
AND TASK MANAGER

Pixi is a modern, fast environment manager that works with Python, R, and many other languages

- Cross-platform: MacOS, Linux, Windows
- Automatically creates a **lock file** - lock files contain the exact versions of all packages installed into an environment, ensuring all users run the exact same versions of every package.

pixi: Reproducible Environments

✗ The Vague Way:

```
```toml
[dependencies]
python = "*"
numpy = "*"
```
```

``python = "*"`` is like saying "I need some food."

Problems:

- Works today, breaks tomorrow
- Different versions on different machines
- "Works on my machine" syndrome

✓ The Locked Way

```
```
pixi.lock contains:
python = "3.11.4"
numpy = "1.24.3"
+ 47 dependencies
```
```

``pixi.lock`` is like saying "I need a pepperoni pizza from Mario's, baked at 5:00 PM."

Benefits:

- ✓ Exact versions locked
- ✓ Same environment everywhere
- ✓ Reproducible in 5 years

Installing pixi

- Full installation docs: <https://pixi.sh>
- In MacOS or Linux

```
curl -fsSL https://pixi.sh/install.sh | bash
```

- In Windows

Install via MSI Installer OR

```
winget install prefix-dev.pixi
```


Why pixi?

Fast, cross-platform, automatic lockfiles

- Works on macOS, Linux, Windows
- Creates pixi.lock automatically
- Supports Python, R, and more

Simple workflow:

```
pixi init          # Create project
pixi add python numpy pandas    # Add dependencies
pixi run python analysis.py     # Run code
```

FAIR Connection:

Standard formats + clear dependencies = Interoperable & Reusable software

Creating a New pixi Project

Create a directory and initialize a **pixi** project:

```
(base) leigh@YRL-ML-DSC101 ~ % mkdir myproject
(base) leigh@YRL-ML-DSC101 ~ %
(base) leigh@YRL-ML-DSC101 ~ % cd myproject
(base) leigh@YRL-ML-DSC101 myproject % pixi init
✓ Created /Users/leigh/myproject/pixi.toml
```

This creates a **pixi.toml** file, which documents your environment.

Example: creating a New pixi Project - pixi.toml

```
(base) leigh@YRL-ML-DSC101 myproject % pixi add python
✓ Added python >=3.14.3,<3.15
(base) leigh@YRL-ML-DSC101 myproject % pixi add numpy
✓ Added numpy >=2.4.2,<3
(base) leigh@YRL-ML-DSC101 myproject %
(base) leigh@YRL-ML-DSC101 myproject % pixi add r
✓ Added r >=4.5,<4.6
(base) leigh@YRL-ML-DSC101 myproject % pixi add r-dplyr
✓ Added r-dplyr >=1.2.0,<2
(base) leigh@YRL-ML-DSC101 myproject %
```


Checking `pixi.toml`

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
[workspace]
authors = ["Leigh Phan <leighphan@ucla.edu>"]
channels = ["conda-forge"]
name = "myproject"
platforms = ["osx-arm64"]
version = "0.1.0"
```

```
[tasks]
```

```
[dependencies]
python = ">=3.14.3,<3.15"
numpy = ">=2.4.2,<3"
r = ">=4.5,<4.6"
r-dplyr = ">=1.2.0,<2"
```

```
~
```

```
~
```

The `pixi.toml` file is now a reproducible record of all dependencies needed for the software.

Running Code with pixi

Run Python code:

```
pixi run python script.py
```

Run R code:

```
pixi run Rscript analysis.R
```

- Every command is executed inside the environment described by **pixi.toml**.
- This makes it easier for others to test, cite, extend, build upon your work.

How Environments Support Citation and Reuse

A reusable research software project contains not only code, but:

- Licensing
- authorship and citation metadata (CITATION.cff)
- version information
- a **documented environment**

Publishing and Citation

When publishing your software, include:

- the `pixi.toml` file
- instructions such as:

```
pixi run python script.py
```

Reuse in practice:

- Including `pixi.toml` in your repository or DOI deposit helps future readers:
 - recreate the execution environment
 - verify results
 - adapt your code for new analyses
 - evaluate whether the software is FAIR (Findable, Accessible, Interoperable, Reusable)

The Citation Problem

Scenario: You publish a paper using someone's code.

✗ Bad Citation: > "We used the analysis script from <https://github.com/jt14den/software-demo>"

Problems:

- URL breaks (link rot)
- No version specified
- No formal credit
- Author's name not visible

The Citation Problem

✓ Good Citation:

Dennis, T. (2025). *Biodiversity Analysis Toolkit* (v0.1.0). Zenodo.
<https://doi.org/10.5281/zenodo.123456>

Why This Works:

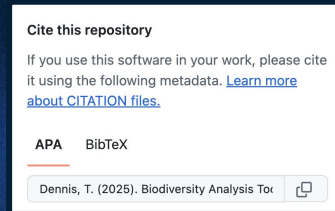
- DOI is permanent
- Version is explicit
- Author gets formal credit
- Appears in citation metrics

The Fix: CITATION.cff file

CITATION.cff: The "Cite Me" Button

```
cff-version: 1.2.0
message: "If you use this software, please cite
it as below."
authors:
  - family-names: "Dennis"
    given-names: "Tim"
    orcid:
      "https://orcid.org/0000-0002-1234-5678"
title: "Biodiversity Analysis Toolkit"
version: 0.1.0
date-released: 2025-01-15
url: "https://github.com/jt14den/software-demo"
```

GitHub creates a "Cite this repository" button automatically!



How to Create a CFF Doc

Use CFF INIT:

- More information on CFF: <https://citation-file-format.github.io/>
- A wizard for creating your own CFF file: <https://citation-file-format.github.io/cff-initializer-javascript/#/>
- Fill out, download and add to your repo

The screenshot shows the 'Basic information' section of the CFF INIT web form. The header includes the 'CFF INIT' logo, a 'REPORT AN ISSUE' link, and a 'DOCUMENTATION' link. A blue button labeled 'Open CFF Preview' is in the top right. On the left, a vertical list of form sections is shown with progress indicators: 'Basic Information required' (active), 'Authors required', 'Identifiers optional', 'Related Resources optional', 'Abstract optional', 'Keywords optional', and 'License optional'. The main content area is titled 'Basic information' and contains instructions: 'Indicate whether your work is a software or a dataset, give it a title, and optionally change the default message.' Below this, there is a 'Type of the work' section with radio buttons for 'Software' (selected) and 'Dataset'. A red-bordered box highlights the 'Title of the software (required)' field, which has a red exclamation mark icon and an information icon. A red error message 'Title cannot be empty.' is displayed below the title field. At the bottom, there is a text area for a 'Personalized message. Leave blank to use default' with an information icon.

CFF INIT [REPORT AN ISSUE](#) [DOCUMENTATION](#)

Basic information [← Open CFF Preview](#)

Indicate whether your work is a software or a dataset, give it a title, and optionally change the default message.

Type of the work ⓘ

☒ Software ☐ Dataset

Title of the software (required) ⓘ

Title cannot be empty.

Personalized message. Leave blank to use default ⓘ

The Link Rot Problem

GitHub URLs are not citations. They break.

Timeline:

2020: Project starts → github.com/jt14den/software-demo

2022: Author changes username → github.com/timdennis/software-demo

2025: Repository archived → "404 Not Found"

Your citation is now dead. 🦴

The Solution: Minting a DOI

DOI = Digital Object Identifier

Think of it as a **QR Code** for your software project.

-  Never breaks (permanent redirect)
-  Points to archived snapshot (Zenodo)
-  Includes metadata (author, version, license)
-  Citable in academic publications

Example: ``https://doi.org/10.5281/zenodo.123456``

Even if GitHub disappears, your DOI still works.

Minting a DOI: The Workflow

Branch: 05-release

Step 1: Tag your release

```
git tag v0.1.0  
git push --tags
```

Step 2: Create GitHub Release

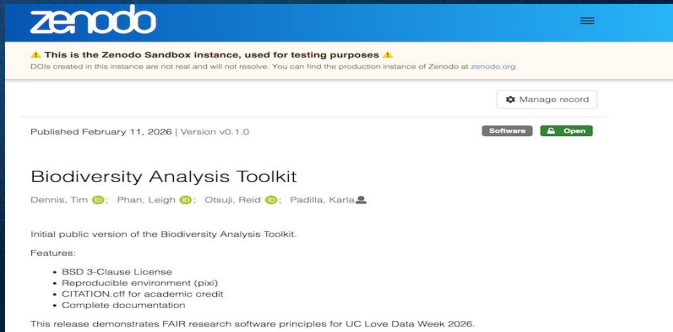
- Go to: Releases → Draft a new release
- Select tag: v0.1.0
- Add release notes

Step 3: Connect Zenodo

- Link GitHub repo to Zenodo
- Zenodo automatically mints DOI for each release
- ****Result:**** Permanent, citable version of your software
-

Steps to Mint DOI

1. Log in to Zenodo with GitHub
2. Enable your repository (toggle ON)
3. Create GitHub Release (tag v1.0.0)
4. Zenodo auto-archives and mints DOI
5. Add DOI badge to your README
6. Update your CFF file with your DOI



Now we have: **LICENSE**,
environment,
CITATION.cff, **DOI**.

But where do people look
first? Your **README**.



Writing a README

Your README is your software's front door.

If users can't understand what it does, how to install it, or how to use it in 30 seconds → they leave.

The Fix:

A structured README makes your software:

- **Findable:** Clear description aids discovery
- **Accessible:** Easy installation removes barriers
- **Reusable:** Usage examples enable adoption



UC OSPO Guide: <https://ucospo.net/oss-resources/template-guides/readme-guide/>

README Structure (UC OSPO Template)

Essential sections to include:

1. **About:** What does this do? (2-3 sentences)
2. **Features:** Key capabilities (reproducible, citable, open source)
3. **Getting Started:** Prerequisites + installation
4. **Usage:** Minimal working example
5. **Citation:** Link to CITATION.cff or DOI
6. **License:** Explicitly state terms
7. **Contact:** How to get help

Don't reinvent the wheel:

<https://github.com/UC-OSPO-Network/templates/blob/main/README.md>

README: Before vs. After

✗ Before (01-start)

```
# Biodiversity Analysis Toolkit
```

A script.

No description. No instructions. No citation. Unusable.

Professional. Citable. Usable.

✓ After (06-metadata)

```
# Biodiversity Analysis Toolkit
Analysis tools for biodiversity research.
## Features
- Reproducible (pixi) • Citable (DOI) • Open source
(BSD-3)
## Getting Started
```bash
pixi install
pixi run python src/analysis.py
```

Citation
[zenodo.org/badge/DOI/10.5281/zenodo.123456.svg]
License
BDS - see LICENSE file
```


README Best Practices

5 Quick Tips:

1. **Clear description** → Answer "What problem does this solve?"
2. **Show, don't tell** → Include code examples
3. **Link metadata** → Add DOI badge, link CITATION.cff
4. **Keep updated** → Refresh when features change
5. **Use the template*** → <https://github.com/UC-OSPO-Network/templates>

Every tip maps to FAIR principles.

Beyond the README

GitHub looks for:

- CONTRIBUTING.md → How to contribute
- CODE_OF_CONDUCT.md → Behavioral standards
- CHANGELOG.md → Version history

Why it matters:

Signals your project is professionally managed and welcoming.

Templates: <https://github.com/UC-OSPO-Network/templates>

Awesome README: <https://github.com/matiassingers/awesome-readme/>

Your FAIR4RS Checklist

What we covered today:

- ✓ **F - Findable:** Added **DOI**, **CITATION.cff**, rich metadata
- ✓ **A - Accessible:** Public **GitHub**, archived on Zenodo
- ✓ **I - Interoperable:** Standard formats (YAML, CFF), documented dependencies (**pixi.toml**)
- ✓ **R - Reusable:** **LICENSE** (BSD-3), **README** with setup, environment reproducibility

Resources

Lesson Materials:

 <https://github.com/carpentries-incubator/research-software-citable-discoverable>

Tools:

- **CITATION.cff** Helper: <https://citation-file-format.github.io/cff-initializer-javascript/>
- **Zenodo**: <https://zenodo.org>
- **Pixi**: <https://pixi.sh>
- **Choose a License**: <https://choosealicense.com>
- **UC OSS-Chart**: <https://security.ucop.edu/files/documents/resources/oss-chart.pdf>
- **UC OSPO License Guide**:
<https://ucospo.net/oss-resources/template-guides/license-guide/>

UC Open Source Program Office (OSPO) Network

 <https://ucospo.net/>

UNIVERSITY
OF
CALIFORNIA



Open Source
Program Office

Ucla
UCSB

UC SANTA CRUZ

UC San Diego

UC Berkeley

UCDAVIS

UC Berkeley **UCDAVIS** *Ucla* UC San Diego UCSB UC SANTA CRUZ

Questions

Thank you!

Tim Dennis • tdennis@library.ucla.edu

Laura Langdon • llangdon@ucdavis.edu

Karla Padilla • kpadilla@ucsd.edu

Leigh Phan • leighphan@library.ucla.edu

Reid Otsuji • rotsuji@ucsd.edu